MILO

MODULAR INTERACTIVE LAUNCHPAD OVERLAY

THE DESKTOP, REIMAGINED.
A persistent, intelligent-feeling Windows companion with a distinctive visual identity.

DOCUMENT	VALUE
Version	2.0 — Full product requirements
Prepared	2 November 2026
Target	Windows desktop application
Product scope	Complete MILO application — all four decks and native integrations
Recommended stack	Tauri 2 • Rust • React • TypeScript • SQLite
Audience	Product owner, Codex, desktop engineers, QA and release engineering

Scope statement. This document specifies the complete intended product, not a reduced prototype. It defines required behavior, architecture, Windows API integrations, data contracts, security, reliability, testing, packaging and acceptance criteria. Implementation may be staged internally, but no major product area is silently excluded from the release target.

Status: authoritative working specification. Where Windows behavior depends on hardware, policy, runtime or provider terms, the requirement explicitly defines graceful degradation instead of promising impossible universal behavior.

Contents

- 1. Product definition and scope
- 2. Product principles and user experience
- 3. Technology decisions and research findings
- 4. System architecture
- 5. Windowing, positioning and interaction model
- 6. Visual design system and motion
- 7. System deck requirements
- 8. Productivity deck requirements
- 9. Dropzone and native file transfer
- 10. Music deck and lyrics
- 11. Milo companion
- 12. Settings, onboarding and lifecycle
- 13. Data model and persistence
- 14. IPC contracts and event model
- 15. Security and privacy
- 16. Performance, reliability and accessibility
- 17. Packaging, updates and release engineering
- 18. Testing and acceptance criteria
- 19. Implementation and delivery plan
- 20. Codex operating instructions
- Appendix A. Proposed project structure
- Appendix B. Source register and research notes

How to read this document

“MUST” denotes a release requirement. “SHOULD” denotes a strong requirement that may be changed only with a documented reason. “MAY” denotes an optional enhancement. Targets are measurable engineering goals, not guarantees; they must be verified on representative hardware.

1. Product definition and scope

MILO is a persistent, top-centre Windows desktop overlay: a compact island attached visually to the top screen edge that expands into a polished interactive deck. It combines system controls, personal productivity utilities, temporary file staging and media controls in one coherent surface.

1.1 Product identity

Name: MILO. **Expansion:** Modular Interactive Launchpad Overlay. **Platform:** Windows first. **Distribution:** downloadable installer, not a website. **Interaction:** hover, click, keyboard shortcut, scroll and drag-and-drop.

1.2 Core product areas

Area	Required capabilities
System deck	Time/date, battery, master volume, mute, brightness, Wi-Fi, Bluetooth, energy-saver status and Windows settings shortcuts.
Productivity deck	Tasks, lists, reminders, timers, alarms, quick notes, recurrence, completion and native notifications.
Dropzone deck	Accept files from Explorer, hold file references, preview metadata, persist stack, reorder, open/reveal/copy paths and drag files out to other apps.
Music deck	Detect Windows media sessions, metadata, artwork, playback state, transport controls, timeline/seek where supported, lyrics and synchronized lyric display where available.
Milo companion	Animated avatar visible only in expanded state; contextual moods and low-cost animation.
Application shell	Collapsed island, expansion, deck navigation, settings, startup, global shortcut, multi-monitor handling, local persistence, installer and updater.

1.3 Explicit non-goals for the initial product

- MILO is not a music streaming service and does not host or distribute audio.
- MILO is not a full Windows settings replacement; unsupported or policy-controlled operations must link to the relevant Windows Settings page.
- MILO does not upload files, notes, tasks or reminders to a cloud service by default.
- MILO does not require a user account, subscription or always-on network connection for core functionality.
- MILO does not execute, modify or delete files merely because they are placed in Dropzone.
- MILO does not promise that a powered-off computer can deliver an alarm while it is off.

1.4 Target operating systems

Primary support is current supported Windows 11 x64 releases, with QA coverage on the current and immediately preceding supported feature release (including 24H2/25H2 where applicable). Windows 10 22H2 is best-effort compatibility only; it is not the product's long-term support baseline. ARM64 is a later validation target unless release hardware and dependencies are confirmed.

1.5 User stories

- As a user, I can glance at the top edge and see the time, battery and relevant active indicators without opening a full app.
- As a user, I can hover or press a shortcut to reveal the island without losing focus in the application I was using.
- As a user, I can change supported Windows controls and see changes made elsewhere reflected in MILO.
- As a user, I can drop files into MILO, switch applications and drag them out again.
- As a user, I can rely on reminders and alarms to persist across MILO restarts, with clear behavior when the PC was asleep or powered off.
- As a user, I can control media from a supported player and see lyrics only when an authorized source provides them.

2. Product principles and user experience

2.1 Product principles

- **One coherent object:** decks share a shell, lighting language, spacing system and navigation behavior.
- **Native where it matters:** Windows APIs perform system operations; web UI is only the presentation layer.
- **Quiet by default:** collapsed state is compact, low-distraction and does not steal focus.
- **Truthful state:** show confirmed OS state, not optimistic UI state that assumes an operation succeeded.
- **Local-first:** personal data remains on the device unless the user explicitly opts into a future sync feature.
- **Graceful degradation:** unavailable hardware, permissions or providers disable only the affected capability.
- **Performance is a feature:** the overlay must not become a persistent battery or CPU tax.

2.2 Primary interaction journey

1. Application launches at login only if the user enabled that preference; otherwise it starts when manually opened.
2. MILO appears collapsed at the top-centre of the selected monitor.
3. Pointer entry into the visible anchor region starts a short configurable hover delay.
4. The shell expands upward and downward from the top attachment, while the avatar appears.
5. The active deck is restored (or a user-selected default deck is shown).
6. Scrolling over the deck content changes decks; scrolling inside a nested scrollable list must not accidentally switch decks.
7. Pointer exit starts a dismissal delay; the interface remains open while a control is being dragged, edited or interacted with.
8. Escape dismisses; a configurable global shortcut toggles or focuses the island.

2.3 Interaction state machine

State	Meaning / exit condition
INITIALIZING	Load config, database, native adapters and notification reconciliation.
COLLAPSED	Compact island; no avatar; no unnecessary animation.
HOVER_PENDING	Pointer entered; activation delay running.
EXPANDING	Window geometry and UI animate into expanded state.
EXPANDED_PASSIVE	Visible, pointer may be outside; dismissal timer active.
EXPANDED_INTERACTIVE	Input, drag, text editing, popover or control interaction active; do not dismiss.
DECK_TRANSITION	Active deck changes; navigation and scroll lock are coordinated.
DRAG_SOURCE	Native OLE file drag initiated; shell must not collapse.
SUSPENDED	Hidden by user, fullscreen policy or system lifecycle.
SHUTTING_DOWN	Persist state, unregister transient handlers and close.

2.4 Focus and dismissal

- Hover expansion MUST NOT activate the window or steal keyboard focus.
- Explicit click or shortcut activation MAY focus MILO so keyboard navigation and text entry work.
- Click-outside dismissal must distinguish clicks inside controls, nested popovers and the native drag operation.
- Escape dismisses transient overlays first (popover, modal, menu), then the deck, then the island.

- When a text field has focus, hover exit MUST NOT collapse the window.
- The app must expose a setting for hover activation, delay, dismissal delay and shortcut behavior.

3. Technology decisions and research findings

The chosen stack balances a highly custom web-rendered interface with native Windows access. Tauri 2 is recommended, but the native Windows boundary is treated as a first-class engineering subsystem rather than an afterthought.

3.1 Recommended stack

Layer	Choice	Reason / constraint
Desktop shell	Tauri 2	Native window lifecycle, packaging, permissions and Rust integration; transparent-window behavior still requires Windows-specific validation.
Backend	Rust	Safe native integration, concurrency, structured services and direct access to Windows APIs via windows crate.
Frontend	React + TypeScript	Reusable deck/component model, typed UI and strong ecosystem.
Build	Vite + pnpm	Fast frontend build and reproducible lockfile.
Animation	Motion for React	Spring transitions and layout animation; keep motion isolated and reduced-motion aware.
State	Zustand	Small focused stores for UI state; native state remains owned by Windows.
Persistence	SQLite	Embedded, transactional, offline-first storage; use migrations and backups.
Windows bindings	windows crate	Rust projections for Win32 and WinRT APIs; isolate unsafe calls in adapter modules.
Notifications	Windows App SDK AppNotificationManager	Current recommended desktop app notification API; scheduled notification behavior has delivery-window limitations.
Icons	Lucide	Consistent line iconography; custom MILO mark and mascot remain bespoke.
Installer	Tauri NSIS setup EXE	Consumer-friendly direct download; MSI optional for managed deployment.
Updater	Tauri updater plugin	Signed update artifacts and explicit update flow.

3.2 Why not Electron?

Electron is a viable alternative with a mature Node ecosystem and extensive native modules. It generally carries a larger runtime footprint and a broader bundled Chromium/Node surface. MILO's always-running overlay makes idle footprint and background efficiency important, so Tauri is the preferred starting point. Electron should be reconsidered only if a required native integration or deployment constraint proves materially easier and safer there.

3.3 Why not pure WinUI 3?

WinUI 3 offers the most direct Windows-native UI and App SDK integration. It is a serious alternative if deep Windows composition and native controls outweigh the value of a React/CSS-driven design system. The current product direction favors a custom-rendered visual surface, so Tauri is selected with a deliberate Windows interop layer. A small Windows App SDK bridge may be used for notification registration/scheduling if direct Rust projection is not robust enough.

3.4 WebView2 runtime

Tauri on Windows uses Microsoft Edge WebView2. Production installation MUST verify the WebView2 Runtime is available. Prefer the Evergreen Runtime so security and web platform updates are serviced by Microsoft. Do not bundle a Fixed Version runtime by default: it materially increases installer size and transfers runtime servicing responsibility to MILO. Provide an understandable recovery path if WebView2 is missing or damaged. [S01]

3.5 Native notification implementation decision

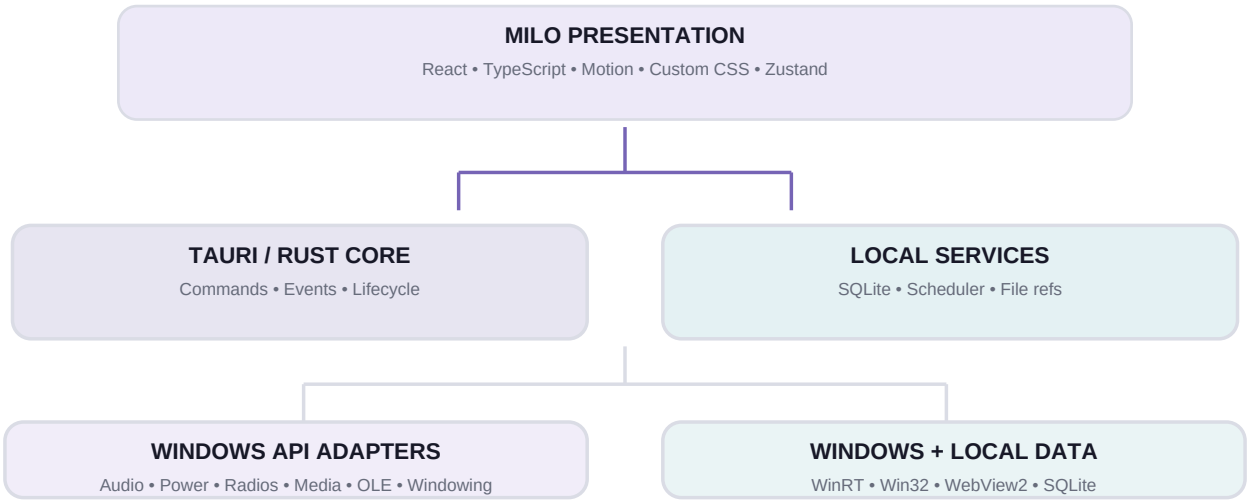
Use Windows App SDK AppNotificationManager for Windows desktop notifications. Microsoft currently recommends it for new Windows App SDK apps and also documents support for unpackaged Win32 desktop apps. Tauri is not itself a Windows App SDK project, so the implementation must validate registration, activation and scheduled notification behavior in a small native proof-of-concept. Preferred integration is direct WinRT projection if reliable; otherwise bundle a minimal signed .NET Windows App SDK notification bridge as an internal helper within the same MILO installer/product. Do not fall back silently to an obsolete toast sample. [S02, S03]

3.6 Research-derived constraints

- Radio state reads are generally available, but changing Wi-Fi/Bluetooth radio state requires a permission request and can still be blocked by hardware or system policy. Confirm final state from the radio event/state, not merely from the request result. [S04]
- Brightness control is hardware-dependent. Internal laptop panels commonly expose WMI brightness methods; external monitors may require DDC/CI and may not support all controls. [S05]
- Global media session APIs expose the current session, available sessions, metadata, playback info and supported transport operations; not every player supports every action. [S06]
- Windows OLE drag-and-drop uses native IDataObject/IDropSource/DoDragDrop contracts. A DOM drag event alone is not an interoperable Windows file drag source. [S07]
- Scheduled app notifications have a five-minute delivery window. If the PC is off beyond that window, the notification may be dropped; reconcile overdue reminders at next app launch and state this limitation clearly. [S08]
- Windows background work can increase battery consumption; hidden/idle MILO should avoid CPU wakeups, timers and unnecessary vsync-driven rendering. [S09]

4. System architecture

4.1 Logical architecture



The frontend renders state and sends intent. The Rust core validates requests, coordinates services and calls native adapters. Windows owns the actual system state. SQLite owns MILO's persistent user data. External providers are isolated behind adapters.

4.2 Architectural boundaries

Layer	Responsibilities	Must not do
Presentation	Render shell, decks, controls, animation, accessibility, local input validation.	Call Windows APIs directly or assume system mutations succeeded.
UI state	Active deck, expanded state, local form state, transient loading/error states.	Become source of truth for battery/audio/radio/media.
Tauri command/event layer	Typed request/response, event delivery, capability enforcement.	Expose arbitrary shell, filesystem or process execution.
Rust application core	Business rules, scheduler coordination, lifecycle, service orchestration, structured errors.	Contain all low-level Win32 code in one monolithic file.
Windows adapters	Win32, COM, WinRT, WMI, OLE and monitor APIs.	Leak COM pointers or platform-specific types into frontend contracts.
Persistence	SQLite migrations, transactions, backups, data validation.	Perform long blocking operations on UI thread.
Provider adapters	Lyrics lookup and future optional integrations.	Couple external service responses directly to UI components.

4.3 Rust Windows adapter strategy

Use the windows crate for Win32 and WinRT APIs. Keep unsafe blocks narrow, document ownership/threading assumptions and expose safe internal interfaces. COM integrations must initialize COM on the correct apartment and release interfaces deterministically. Use a dedicated STA thread/message pump for OLE drag-source work. Do not run COM-bound operations indiscriminately on Tokio worker threads. [S10]

4.4 Optional helper process

A helper is permitted only for a Windows App SDK capability that is not robustly callable from the Rust process, particularly AppNotificationManager registration/activation. If required, it must be a narrowly scoped notification bridge, installed and versioned with MILO, communicating over a local authenticated/named-pipe or stdio protocol. It must not become a second user-facing app, expose a network listener or duplicate core business logic.

4.5 Failure isolation

- Each Windows integration is an independently initialized service with explicit capability status.
- Failure to initialize Bluetooth must not prevent battery, audio, Dropzone or productivity features from loading.
- Each command returns a typed result with a stable error code and recoverability flag.
- Native callbacks must be detached during shutdown and device/session replacement.
- The UI must show stale/unavailable status when a native event stream disconnects.

5. Windowing, positioning and interaction model

5.1 Native window requirements

- Borderless window with no standard title bar or resize frame.
- Transparent native window background; web content background must also remain transparent outside visible surfaces.
- Always-on-top behavior, configurable where practical.
- Top-centre positioning relative to a selected monitor's work/display area.

- Dynamic resize/reposition during expansion without a visible jump.
- No taskbar button by default; tray icon may be provided for access/settings/exit.
- Correct behavior across DPI changes, monitor topology changes, sleep/resume and display disconnect.
- No interaction with secure desktop, UAC prompts or lock screen; hide/suspend when appropriate.

5.2 Geometry

Parameter	Initial design target	Notes
Collapsed width	220–290 logical px	Responsive to status indicators; avoid crowding.
Collapsed height	42–52 logical px	Top attachment tab/pill.
Expanded width	560–740 logical px	Scale down for smaller work areas.
Expanded height	430–600 logical px	May adapt to display height; must not cover taskbar.
Corner radius	20–32 px	Organic soft geometry.
Expansion duration	250–450 ms	Spring-based; respect reduced motion.
Hover delay	100–220 ms default	Configurable; avoid accidental activation.
Dismiss delay	350–700 ms default	Longer while interaction is active.

These are starting design tokens. Actual geometry must be tested at 100%, 125%, 150%, 175% and 200% scaling, including small laptop displays and multi-monitor mixed-DPI configurations.

5.3 Click-through and hit-testing

A transparent rectangular window can intercept pointer input even where no visible UI exists. Do not solve this by setting the entire window click-through while the user still needs to interact with the pill. Implement region-aware native hit-testing (for example, handling WM_NCHITTEST and returning HTTRANSPARENT for truly empty regions) or a rigorously tested equivalent. Recompute interactive regions after every resize/animation geometry change. Tauri's whole-window ignore-cursor-events behavior is not, by itself, sufficient for per-region hit testing.

5.4 Window composition and transparency

- Use a transparent WebView/window surface and custom CSS surfaces for smoked glass, gradients and borders.
- Use backdrop-filter only where supported and profiled; provide a visually acceptable fallback.
- Avoid requiring Acrylic/Mica to achieve the brand identity; native effects are optional enhancement, not a core dependency.
- Test resize, minimize/restore, topmost changes and monitor changes for composition artifacts.
- Do not render a large transparent full-screen window unless technically necessary; keep the native bounds close to the island to reduce hit-testing and composition cost.

5.5 Fullscreen and z-order policy

Provide a policy to auto-hide when a fullscreen application or exclusive fullscreen game is active. Never force the overlay above secure desktop surfaces. Respect user preference for always-on-top but provide a quick temporary hide action. Do not interfere with screen sharing or remote desktop sessions; test and document behavior.

5.6 Global shortcut

Default suggestion: Ctrl + Shift + Space. Register through the Tauri global-shortcut plugin or native RegisterHotKey where appropriate. Detect registration conflicts, show a recoverable warning and allow rebinding. Shortcut behavior should be configurable: toggle, show/focus, or hide. [S11]

5.7 Multi-monitor policy

- Default to the primary monitor unless user selects another.
- Persist selected monitor by stable display identity where possible, with fallback to primary.
- Position relative to the monitor work area and respect display cutouts/taskbar arrangements where detectable.
- Handle monitor removal by relocating to primary and updating stored selection.
- Use per-monitor DPI-aware coordinates and explicitly convert logical/physical pixels.

6. Visual design system and motion

6.1 Art direction

Direction: rich, elegant futurism. **Appearance:** predominantly dark with luminous accents. **Geometry:** soft organic curves. **Material:** smoked translucent glass layered with pearlescent gradients, restrained specular highlights and subtle ambient bloom. MILO must feel like a designed object, not a generic glass dashboard.

6.2 Design tokens

Token	Suggested value	Usage
Background	#11121B / #171827	App canvas and deep surfaces.
Surface	rgba(32, 34, 49, 0.78)	Main smoked-glass panel.
Surface elevated	rgba(48, 49, 68, 0.72)	Cards, popovers and nested controls.
Primary accent	#A995F2	Focus, active navigation and selected controls.
Secondary accent	#79D1DE	System status and cool highlights.
Pearl highlight	#E9E3FF	Edge reflection and high-value emphasis.
Text primary	#F4F2FA	Primary labels.
Text secondary	#AAA9BA	Supporting information.
Border	rgba(235,230,255,0.14)	Glass edge and separators.

6.3 Component rules

- Build custom GlassPanel, GlassButton, Toggle, Slider, SegmentedControl, Tooltip, Popover, Card and StatusIndicator components.
- All controls must have hover, pressed, focused, disabled, loading and error states.
- Sliders must support pointer, keyboard and accessible value announcements.
- Avoid default HTML select/checkbox styling in primary UI.
- Keep a consistent spacing scale (4/8/12/16/20/24 px) and typography hierarchy.
- Use system Segoe UI/Segoe UI Variable as the baseline; bundle another font only with verified redistribution rights.

6.4 Motion rules

Motion	Behavior
Expand/collapse	Spring-based scale/height/opacity transition; visually anchored to top edge.
Deck change	Short directional movement and opacity crossfade; preserve shared shell geometry.
Avatar entrance	Small float/scale/fade timed with expansion; never block interaction.
Control feedback	Immediate thumb/value feedback; native result reconciles afterward.
Ambient lighting	Slow, subtle gradient drift only while expanded; no continuous heavy shader.
Reduced motion	Disable nonessential drift and replace spring transitions with short fades or instant changes.

6.5 Visual quality gates

- No full-window blur or bloom that causes measurable idle GPU load.
- Text contrast remains readable over all glass surfaces.
- Active deck identity is immediately clear without oversized navigation.
- No visual jump between collapsed and expanded anchor.
- Milo is visually integrated but never obscures controls.
- Screenshots at 100%, 150% and 200% scaling must be reviewed before release.

7. System deck requirements

7.1 Time and date

- Display local time and date; support 12/24-hour preference and locale-aware formatting.
- Respect Windows regional settings by default; allow optional format override.
- Update at the next minute boundary where seconds are not displayed; avoid a permanent high-frequency timer.
- Support time-zone and daylight-saving changes without restarting.

7.2 Battery and power state

Use Win32 GetSystemPowerStatus for AC/battery state, battery percentage and system battery-saver status where exposed. Subscribe to power broadcasts where practical and use conservative fallback refresh. A desktop without a battery must show a neutral 'No battery' state, not 0%. Unknown percentage must be represented as unavailable. [S12]

Field	UI behavior
Battery percent	0–100% or unknown; never fabricate a value.
AC state	Plugged in / on battery / unknown.
Charging	Charging / not charging / unknown based on available signals.
Battery saver	On/off/unknown; label as Windows Battery Saver or Energy Saver according to OS naming.
Low battery	Configurable warning threshold; do not duplicate Windows warnings excessively.

7.3 Master volume and mute

Use Windows Core Audio MMDevice and IAudioEndpointVolume for the default render endpoint. Register endpoint-volume callbacks and device notifications so changes made through hardware keys, Windows quick settings or another app update MILO. Set volume using scalar 0.0–1.0; map to UI percentage. Mute/unmute must reflect actual endpoint mute state. Handle device changes and endpoint invalidation. [S13]

- Display current output endpoint name optionally; avoid overloading collapsed state.
- Volume slider updates immediately but debounces/coalesces native writes during drag.
- On drag end, read back actual value and reconcile UI.
- Provide clear error if no output device is available.
- Optional later control: choose output device; not required for initial baseline.

7.4 Brightness

Implement a capability-based brightness adapter. For supported internal panels, query WMI root\wmi WmiMonitorBrightness and invoke WmiMonitorBrightnessMethods.WmiSetBrightness. For external monitors, optionally use Monitor Configuration API/DDC-CI only after confirming monitor support. Do not use gamma ramps as a fake substitute for hardware brightness. [S05]

- Enumerate displays and identify controllable panels.
- Show brightness control only for supported displays or show disabled state with explanation.
- Handle multiple displays individually; default to the primary/internal display.
- Read back brightness after setting; drivers may clamp or reject values.
- Avoid blocking the UI while WMI or DDC/CI calls run.
- Provide 'Open display settings' fallback.

7.5 Wi-Fi

Use Windows.Devices.Radios where supported to enumerate Wi-Fi radios and read radio state. Request radio access only when the user explicitly attempts to toggle. Check permission status, hardware capability and final StateChanged event. The app must not assume that an accepted request means the radio reached the requested state. Network connection details (SSID) are optional and require a separate, privacy-reviewed adapter; the initial requirement is radio status and supported toggle. [S04]

7.6 Bluetooth

Use the Windows Radio API for Bluetooth radio status and permitted on/off control. Pairing, device discovery and profile management remain in Windows Settings for the initial release. Provide an explicit route to Bluetooth settings. Handle airplane mode, hardware kill switches, policy restrictions and permission denial.

7.7 Energy saver / power mode

Display the OS battery-saver/energy-saver state where available through power status. Do not claim to control the full Windows power mode slider unless a supported API has been proven. The initial control may open Windows Power & battery settings. Clearly distinguish battery saver from Windows power mode (Best power efficiency/Balanced/Best performance).

7.8 System deck acceptance

- Battery accurately reflects system state and handles desktops.
- Volume control changes default output endpoint and reflects external changes.
- Brightness changes actual supported display brightness; unsupported cases are explicit.
- Wi-Fi/Bluetooth status is accurate; toggles require permission and confirm final state.
- Energy saver status is not confused with performance mode.
- No system operation requires administrator elevation for ordinary supported use.
- All system calls are asynchronous from the UI perspective and return typed errors.

8. Productivity deck requirements

8.1 To-do lists

- Create, edit, complete, reopen and delete tasks.
- Support multiple named lists, task title and optional description.
- Support due date/time, priority and completion timestamp.
- Sort by due date, priority, creation time and manual order.
- Filter all/open/completed/overdue.
- Provide quick-add from any deck through a compact capture interaction.
- Confirm destructive bulk deletion; ordinary completion should be one action.

8.2 Reminders

- Create one-time and recurring reminders with title, optional detail, date/time and enabled state.
- Support snooze, dismiss, complete and edit.
- Support recurrence presets (daily, weekdays, weekly, monthly) and a defined subset of RFC 5545 RRULE if advanced recurrence is offered.
- Store timezone context and handle daylight-saving/time-zone changes.
- Rescheduling or deleting a reminder must update/cancel the corresponding OS scheduled notification.
- If the machine was powered off during the delivery window, reconcile overdue reminders on next launch according to a user-configurable missed-reminder policy.

8.3 Timers

- Create named timers with duration; allow multiple concurrent timers.
- Start, pause, resume, reset and delete.
- Use an absolute deadline plus monotonic clock calculations while running; do not decrement a counter once per second as the source of truth.
- Persist running/paused state and reconcile after app restart.
- Provide an in-app completion state and optional Windows notification.
- Define sleep behavior: elapsed wall-clock time continues by default; offer setting if users prefer pause-on-sleep.

8.4 Alarms

- Create one-time and recurring alarms with label, time, days and enabled state.
- Support configurable sound, snooze interval and dismiss.
- Use scheduled Windows notifications for delivery when MILO is not visible.
- Document that Windows cannot deliver while the computer is powered off; notifications may be dropped if the machine is off beyond the platform delivery window.
- On next launch, show overdue alarm summary only according to user policy; avoid firing a large backlog of stale alarms.

8.5 Notes and quick capture

- Create, edit, pin and delete local notes.
- Autosave with short debounce and explicit saved indicator.
- Support plain text in v1; rich text/markdown can be optional.
- Escape rendered content and never execute pasted HTML.

- Provide quick capture from the collapsed island only if it can be done without making the island crowded.

8.6 Notifications subsystem

Use Windows App SDK AppNotificationManager for app notifications. Register activation handling before registration where required. Use scheduled notifications for reminders/alarms and local immediate notifications for timer completion. Keep the notification identifier linked to the reminder/alarm row. Provide permission/notification settings guidance and respect Windows notification settings. [S02, S03, S08]

8.7 Scheduling reliability

The scheduler must persist due timestamps in SQLite and reconcile OS schedules at startup. Scheduled notifications are a delivery mechanism, not the database of record. On every launch/resume/time-zone change, compare active reminder rows with scheduled IDs, repair missing schedules and apply the missed-event policy. Scheduled Windows notifications have a five-minute delivery window; if the computer is off longer, the notification can be dropped. Do not market MILO as a guaranteed alarm clock when the PC is off. [S08]

9. Dropzone and native file transfer

9.1 Purpose

Dropzone is a temporary holding shelf. Users collect files from Explorer or other applications, switch to another app, then drag the files out of MILO. It stores references by default, not duplicate file contents.

9.2 Intake requirements

- Accept one or multiple files from Windows Explorer through native drop handling.
- Accept directories only if recursive handling and safety rules are defined; default v1 may reject directories with a clear message.
- Display name, extension, file size, source path availability and added time.
- Generate thumbnails asynchronously for supported image types; do not decode untrusted files on UI thread.
- Support reordering, individual removal and clear-all.
- Persist references across app restarts.
- Mark missing/moved/inaccessible file references without crashing.

9.3 Native drag-out — mandatory engineering requirement

Windows desktop interoperability uses OLE drag-and-drop. A source must provide an IDataObject and IDropSource and call DoDragDrop; OleInitialize is required. For file paths, provide the appropriate shell file-drop data format (CF_HDROP) and advertise copy semantics by default. Implement a genuine native drag source; an HTML draggable element or Tauri's inbound file-drop event does not satisfy this requirement. [S07]

- Run OLE operations on an STA thread with a message pump; never call DoDragDrop from an arbitrary async worker.
- Provide an appropriate drag image/visual cue where feasible.
- Support copy by default; move must not delete originals unless explicitly designed and confirmed.
- Keep the island expanded and prevent hover dismissal while drag is active.
- Test drag-out to Windows Explorer, desktop, Outlook and at least one browser upload target.
- On drop completion, retain or remove the Dropzone reference according to the user's chosen behavior; never delete the original file.

9.4 Reference model and file lifecycle

Field	Purpose
id	Stable UUID for the Dropzone entry.
canonical_path	Normalized local path; access controlled.
display_name	Name shown in UI.
extension / mime_guess	Display/preview classification; never trusted for execution.
size_bytes	File size at intake; refresh if needed.
added_at_utc	Timestamp of addition.
last_verified_at	Last time existence/access was checked.
status	available, missing, inaccessible, unsupported or stale.
sort_order	User-defined stack order.

9.5 File security

- Never execute, preview-run or shell-open automatically on drop.
- Do not modify original file contents.
- Do not delete originals when clearing Dropzone.
- Avoid following symlinks/reparse points blindly; define path normalization and access checks.
- Do not expose arbitrary file paths to remote content.
- Use OS file permissions; avoid copying file contents into app storage unless user explicitly chooses 'copy into MILO'.
- If a file is dragged out, provide the original path via native shell data object and let the target app negotiate the operation.

9.6 Acceptance criteria

- Multi-file drag-in works from Explorer.
- Stack persists after restart.
- Missing references are identified.
- Open/reveal/copy path actions work.
- Native drag-out works to Explorer and external targets.
- Drag operations do not collapse the island or deadlock the UI.
- Removing a reference never deletes the source file.

10. Music deck and lyrics

10.1 Media session integration

Use GlobalSystemMediaTransportControlsSessionManager to enumerate sessions and obtain the system current session. Subscribe to current-session and session-list changes; subscribe to per-session media-property, playback-info and timeline events. Request only operations supported by the selected session. [S06]

Capability	Requirement
Session selection	Default to Windows' current session; allow choosing among available sessions.
Metadata	Track title, artist, album and optional album track number.
Artwork	Retrieve thumbnail where available; cache with size/eviction limits.

Capability	Requirement
Playback	Show playing/paused/stopped/unknown state.
Controls	Play/pause, previous, next; only show/enable actions supported by session.
Timeline	Position/duration and seek when supported by playback controls and session.
Session changes	Update UI when player starts/stops, track changes or current session changes.
Empty state	Elegant idle state when no media session is active.

10.2 Session behavior

- Do not assume only one player is active.
- Allow the user to pin a preferred player; otherwise use the system current session.
- If a session disappears during a command, return a recoverable session-unavailable error.
- Do not claim seek support unless the session reports the relevant playback capability.
- Avoid excessive metadata polling; use Windows events and refresh on session changes.

10.3 Lyrics provider

Lyrics are not supplied by GSMTC. They require a separate provider adapter. LRCLIB is a plausible initial provider: its current documentation describes anonymous search/get endpoints, plain and timestamped synced lyrics, and asks clients to identify themselves with a User-Agent and respect rate limits. It is not a guarantee of rights to redistribute lyrics. Before public/commercial distribution, review provider terms, content licensing, attribution and caching permissions. [S14]

10.4 Lyrics matching and display

- Query by track title and artist; include album and duration where available to improve matching.
- Validate returned metadata against current track before displaying.
- Prefer synced lyrics; fall back to plain lyrics; otherwise show 'Lyrics unavailable'.
- Parse LRC timestamps defensively and handle malformed lines.
- Highlight current line using session timeline and a monotonic interpolation clock between timeline updates.
- Re-sync when timeline properties change, seek occurs, playback pauses or track changes.
- Provide manual scrolling without fighting automatic scroll; resume-follow control.
- Respect provider rate limits and Retry-After; cache only within provider terms.
- Do not scrape DRM-protected streaming services or bypass access controls.

10.5 Lyrics privacy

Send only the minimum track metadata needed to query lyrics. Do not send listening history, user identity or unrelated file data. Explain that lyrics lookup uses an external service and provide an opt-out setting.

11. Milo companion

11.1 Visibility and role

- Avatar appears only in expanded state and is hidden in collapsed state.
- Avatar is a contextual decorative companion, not an independent pet window.
- Must never obstruct controls, text or accessibility focus.
- Can react to active deck and selected events but must not interrupt the user.

11.2 Mood model

Mood	Trigger / presentation
idle	Default gentle float and breathing.
curious	Pointer proximity or opening a new deck.
focused	User actively working in Productivity.
listening	Music deck has active playback.
happy	Task completion or successful small action.
celebrating	Explicit milestone; brief, restrained animation.
sleepy	Optional idle state after prolonged inactivity.

11.3 Implementation

Implement as a reusable React component driven by a typed mood state. Start with custom SVG/layered vector artwork and CSS/Motion animation. Consider Rive only if expressive animation needs justify the dependency and its runtime/licensing. Do not add a 3D engine for the initial product. Respect reduced-motion settings and stop animation when hidden.

11.4 Art direction

Milo should feel like a small pearlescent, soft-edged presence that belongs to the MILO material system: expressive but understated, with a limited number of clear moods. Its silhouette, eye treatment and animation should be custom and recognizable, not a generic emoji or stock mascot.

12. Settings, onboarding and lifecycle

12.1 Settings categories

Category	Settings
General	Launch at login, start collapsed, selected monitor, always-on-top policy, exit behavior.
Interaction	Hover enabled, hover delay, dismiss delay, global shortcut, click-outside, scroll sensitivity.
Appearance	Accent intensity, transparency, animation intensity, reduced motion, compact/expanded scale.
System	Default output device (if implemented), brightness target display, radio permissions/status.
Productivity	Default reminder behavior, snooze, alarm sound, missed-event policy, timezone.
Music	Preferred media session, lyrics provider, lyrics opt-out, synced/plain preference.
Privacy	External lyrics lookup, diagnostics consent, clear cached artwork/thumbnails.
About	Version, licenses, update check, logs export, privacy policy, support links.

12.2 First-run onboarding

1. Welcome to MILO and short explanation of the top-edge island.
2. Choose hover behavior and default shortcut; offer sensible defaults.
3. Ask whether MILO should launch at Windows startup; default off unless explicitly opted in.
4. Explain that system-control availability depends on device hardware and Windows permissions.
5. Request radio permission only when the user first toggles Wi-Fi/Bluetooth, not during onboarding.
6. Explain external lyrics lookup before first request.

7 Offer a short interaction tour that can be skipped.

12.3 Startup and single instance

- Single-instance enforcement: second launch signals existing process and optionally expands/focuses it.
- Startup launch is opt-in and can be disabled in Settings.
- Start collapsed; do not open the full deck on login.
- Tray icon provides show/hide, settings, pause hover and quit actions.
- Graceful shutdown persists data and unregisters callbacks.
- If a crash occurs, recover database transactionally and offer log export.

12.4 Sleep, resume and session changes

On resume, refresh system state, re-enumerate devices/sessions as needed, reconcile timers and notifications, re-check display geometry and re-register any native callbacks that were invalidated. Avoid a storm of stale notifications after resume.

13. Data model and persistence

13.1 Storage location

Use the per-user application data directory, not the installation folder. Proposed location: %APPDATA%/MILO/ (or Tauri app-data resolver). Keep database, cache, logs and settings separated. Installation/uninstallation must not silently delete user data; offer a separate explicit data-removal action.

13.2 SQLite configuration

- Use SQLite with WAL mode where appropriate, foreign keys enabled and a bounded busy timeout.
- All schema changes use versioned migrations.
- Use transactions for related state changes.
- Perform database work off the UI thread.
- Back up database before destructive migrations.
- Run integrity checks in diagnostics/recovery path.
- Avoid storing large artwork blobs in the main database; use cache files keyed by content hash.

13.3 Proposed schema

Table	Key fields
tasks	id, list_id, title, description, priority, due_at_utc, timezone_id, completed_at_utc, sort_order, created_at, updated_at
task_lists	id, name, color_token, sort_order, created_at, updated_at
reminders	id, title, body, scheduled_at_utc, timezone_id, recurrence_rule, enabled, status, notification_id, created_at, updated_at
timers	id, name, duration_ms, state, started_at_utc, deadline_at_utc, remaining_ms, created_at, updated_at
alarms	id, title, local_time, timezone_id, recurrence_rule, enabled, sound_id, snooze_minutes, notification_id
notes	id, title, content, pinned, created_at, updated_at
dropzone_items	id, canonical_path, display_name, extension, mime_guess, size_bytes, added_at_utc, last_verified_at, status, sort_order
settings	key, value_json, updated_at
notification_schedule	id, entity_type, entity_id, platform_notification_id, scheduled_for_utc, status, updated_at

13.4 Time correctness

- Store absolute instants in UTC; store the relevant Windows timezone identifier for recurring local-time rules.
- Never assume local time is UTC.
- Handle daylight-saving gaps (nonexistent local time) and overlaps (ambiguous local time) with a documented policy.
- For recurring reminders, calculate the next occurrence from local calendar rules, not by adding fixed 24-hour intervals.
- Timers use deadline timestamps and monotonic elapsed-time calculations while running.

13.5 Data retention and backup

Keep a rolling small number of database backups before migrations. Provide a user-visible export/backup option for tasks, reminders and notes (JSON/CSV where suitable). Do not include file contents in Dropzone export; export references only and warn that paths may be machine-specific. Cache cleanup must never delete user originals.

14. IPC contracts and event model

14.1 Command design

Use Tauri commands for explicit request/response operations and events for ongoing native state changes. All payloads are serializable, versionable and typed. Commands validate input and return a structured result. Avoid giant 'do everything' commands.

14.2 Result envelope

```
type MiloResult<T> =
  | { success: true; data: T }
  | { success: false; error: {
      code: string;
      message: string;
      recoverable: boolean;
      retryAfterMs?: number;
    } };
```

14.3 Initial command catalogue

Domain	Commands
Window	get_window_state, set_expanded, set_selected_monitor, get_displays
System	get_system_snapshot, get_battery, get_volume, set_volume, set_mute, get_brightness_targets, set_brightness, get_radio_states, set_radio_state, open_windows_settings
Media	list_media_sessions, get_current_session, select_media_session, control_media, seek_media
Dropzone	list_dropzone, add_file_references, remove_dropzone_item, reorder_dropzone, reveal_file, open_file, start_native_file_drag
Productivity	create_task, update_task, complete_task, create_reminder, update_reminder, delete_reminder, create_timer, control_timer, create_alarm, update_alarm, create_note
Settings	get_settings, update_settings, set_startup_enabled, register_shortcut
Diagnostics	get_app_version, export_logs, run_health_check

14.4 Event catalogue

Event	Payload summary
system.snapshot.changed	Battery, AC, saver, volume, mute and capability flags.
audio.endpoint.changed	Endpoint ID/name/default status.
audio.volume.changed	Endpoint ID, scalar, mute, source.
power.status.changed	Battery/AC/saver state.
radio.state.changed	Radio kind, state, access status.
display.topology.changed	Display IDs, bounds, DPI and primary flag.
media.sessions.changed	Available session IDs and selected session.
media.session.changed	Metadata, playback, timeline and supported actions.
dropzone.changed	Changed item IDs and stack count.
productivity.notification.activated	Notification ID and action.
app.lifecycle.changed	Suspended/resumed/shutdown state.

14.5 Frontend state ownership

Use focused Zustand stores: island, deck, system, media, dropzone, productivity, mascot and settings. UI stores may hold last known native state plus freshness timestamp. Native system state remains authoritative. A command result must not be treated as final until success is confirmed; use optimistic interaction only with rollback/reconciliation.

15. Security and privacy

15.1 Tauri capability model

- Define least-privilege Tauri capabilities for the main window and any helper window.
- Expose only required commands and plugin permissions.
- Do not grant broad filesystem access to the WebView.
- Do not expose arbitrary shell execution, process spawning or unrestricted URL opening.
- Keep native operations behind validated command handlers.
- Use a restrictive Content Security Policy and avoid remote scripts.

15.2 Input and file safety

- Validate all frontend inputs again in Rust.
- Treat file extensions/MIME guesses as untrusted.
- Escape task, note, reminder and lyrics content before rendering.
- No automatic file execution or macro invocation.
- Avoid path traversal and reparse-point surprises.
- Do not permit remote content to invoke native commands.

15.3 Local data protection

Use per-user Windows ACL-protected app-data storage. SQLite is not encrypted by default; do not claim database encryption. If protecting selected sensitive values becomes necessary, use Windows DPAPI or a vetted encryption design and document recovery implications. No passwords or API secrets are required for core MILO.

15.4 Network policy

- Core functionality works offline.
- Only lyrics lookup and signed update checks require network access in the initial design.
- No analytics/telemetry by default; any diagnostics collection must be opt-in and disclosed.
- Use HTTPS and validate update signatures.
- Do not send file paths, notes, tasks or reminders to external services.
- Respect lyrics provider rate limits, attribution and terms.

15.5 Threat model

Threat	Mitigation
Malicious file dropped	No auto-execution; metadata-only preview; safe thumbnail pipeline.
Compromised lyrics response	Treat as untrusted text; sanitize; strict response size/time limits.
WebView command abuse	Tauri capabilities, narrow command surface, validation and CSP.
Update tampering	Signed update artifacts and verification before installation.
Path disclosure	No external file-path transmission; minimal logs; local-only storage.
Privilege escalation	Run as standard user; never auto-elevate.
Data corruption	SQLite transactions, migrations, backups and recovery path.

16. Performance, reliability and accessibility

16.1 Performance budgets

Metric	Initial target / measurement
Cold start to usable collapsed island	≤ 3.0 seconds on representative Windows 11 laptop; measure release build.
Warm activation	≤ 250 ms from shortcut/hover trigger to visible response.
Expansion	250–450 ms animation; no dropped-frame burst on target hardware.
Deck switch	Perceived response < 150 ms for cached content.
Idle CPU	Target < 0.5% average over a 5-minute idle trace after settling; investigate periodic spikes.
Idle memory	Target working set ≤ 180 MB; expanded typical ≤ 250 MB. Validate across WebView2 versions.
UI frame time	Aim for 60 FPS (16.7 ms frame budget) on representative hardware.
Native system update	Event-driven where possible; avoid UI polling loops.
Crash recovery	Recover last committed user data; no silent database reset.

Targets are not claims about guaranteed resource usage. WebView2 runtime, GPU, display scale, animation and hardware integrations affect measurements. Establish a baseline on a clean Release build and track regressions. Microsoft recommends measuring startup, responsiveness, memory and background power use rather than optimizing from assumptions. [S09, S15]

16.2 Background efficiency

- When collapsed, stop ambient animation and expensive visual effects.
- Use event subscriptions for audio, media, radio and power where available.
- Use low-frequency fallback polling only for APIs without notifications.
- Do not run a 60 Hz render loop while collapsed or hidden.
- Do not poll the database or filesystem continuously.
- Cache artwork/thumbnails with a bounded LRU policy.
- Pause nonessential work on battery saver if user enables efficiency mode.

16.3 Reliability

- Every adapter has initialization, ready, unavailable and failed states.
- Use timeouts for native calls and provider requests.
- Retry only transient errors with bounded backoff.
- Honor Retry-After for lyrics service.
- Never block the UI thread on WMI, COM, disk or network work.
- Persist state before shutdown and after user-visible mutations.
- Add crash logs with privacy-safe redaction.

16.4 Accessibility

- Full keyboard navigation and visible focus indicators.
- Accessible names and value announcements for all controls.
- Logical tab order across decks.
- High-contrast/contrast-safe mode or Windows high-contrast compatibility.

- Reduced-motion support.
- Text scaling and display scaling validation.
- Do not communicate state through color alone.
- Tooltips supplement, never replace, accessible labels.

16.5 Localization readiness

Externalize user-facing strings from the start. Format time, date, numbers and file sizes using locale-aware APIs. Layout must tolerate longer translated strings. English is the initial release language; architecture should not hardcode English grammar into data models.

17. Packaging, updates and release engineering

17.1 Distribution format

Primary consumer installer: MILO-Setup.exe using Tauri's NSIS bundler. MSI may be produced as a secondary enterprise/managed-deployment artifact. Installer must create Start Menu shortcut, register application identity/notification assets as needed, configure uninstaller and check WebView2 prerequisite. [S16]

17.2 Installer behavior

- Install per-user by default where supported; do not require admin unless a documented installer operation requires it.
- Offer optional launch-at-login selection; default must be explicit and reversible.
- Do not overwrite or delete user database during upgrade.
- Uninstall application binaries and registered components; retain user data by default with clear instructions for full removal.
- Provide clean upgrade from previous versions.
- Validate WebView2 Evergreen availability and show recovery guidance if absent. [S01]

17.3 Code signing

Sign the installer and executable with a valid code-signing certificate or managed signing service. Protect private signing keys. Verify signatures in CI/release pipeline. Do not distribute unsigned public builds as production releases; SmartScreen reputation and device policy may still affect first-run experience. [S17]

17.4 Automatic updates

- Use Tauri updater with signed artifacts and a trusted update manifest.
- Check on launch and optionally on a user-configurable interval; avoid frequent network checks.
- Show version, release notes and restart requirement.
- Never install an update whose signature cannot be verified.
- Preserve database and settings.
- Support rollback/recovery plan for failed update.
- Provide manual download/update route.

17.5 Release pipeline

1. Run formatting, linting, type checks and Rust tests.
2. Run frontend unit tests and backend unit/integration tests.
3. Build signed Release artifacts on controlled Windows CI.
4. Run installer smoke test in clean Windows VM.

5 Verify upgrade path from previous stable version.

6 Verify notification registration, startup and uninstaller.

7 Generate checksums, SBOM/dependency inventory and release notes.

8 Publish installer, update manifest and privacy/license notices.

18. Testing and acceptance criteria

18.1 Test layers

Layer	Tools / scope
Frontend unit	Vitest + React Testing Library: state, controls, deck switching, empty/error states.
Rust unit	Cargo test: models, validation, scheduler, path handling, persistence.
Integration	Windows test harness: Core Audio, power, radios, GSMTC, WMI, notifications.
Desktop UI	Windows UI automation for window geometry, focus, shortcut and click-through.
Drag/drop	Manual + automated OLE tests with Explorer and target applications.
Performance	Release-build traces, CPU/memory/frame time and battery impact.
Security	Dependency audit, Tauri capability review, CSP review, file/lyrics input fuzzing.

18.2 Hardware test matrix

Test environment	Required validation
Windows 11 laptop, internal panel	Battery, AC state, internal brightness, sleep/resume.
Windows 11 desktop, no battery	No-battery state; system deck layout.
Laptop + external display	Mixed DPI, monitor selection, DDC/CI support/fallback.
Wi-Fi/Bluetooth machine	Radio enumeration, permission prompts, policy/hardware denial.
Multiple audio endpoints	Default endpoint changes, volume callbacks, device removal.
Multiple media players	Session selection, metadata, supported actions and session loss.
Single and multi-monitor	Top-centre geometry, scaling, disconnect and reconnect.
Battery saver enabled	Reduced background work and correct saver status.
Clean VM / WebView2 absent	Installer prerequisite detection and recovery.

18.3 Release acceptance checklist

- Island: collapsed/expanded states, top-centre anchoring, upward/downward visual expansion, focus rules, click-through, global shortcut and multi-monitor behavior pass.
- Visual: dark luminous identity, soft geometry, consistent custom controls, smooth motion, reduced-motion mode and mascot behavior pass.
- System: battery, volume, brightness capability handling, Wi-Fi, Bluetooth and Energy Saver state are truthful and recoverable.
- Productivity: tasks, notes, reminders, timers, alarms, recurrence, restart persistence, notification reconciliation and missed-event policy pass.
- Dropzone: multi-file intake, persistence, missing-file state, open/reveal, native OLE drag-out and no-original-deletion guarantee pass.

- Music: session detection, metadata, artwork, controls, seek capability, session switching, lyrics matching and provider failure behavior pass.
- Security: no arbitrary shell exposure, least-privilege capabilities, restrictive CSP, no automatic file execution, signed updates pass.
- Release: installer, uninstaller, upgrade, code signing, WebView2 check, startup preference and data retention pass.

18.4 Definition of done

A feature is done only when it has: implemented UI, backend/native behavior where applicable, typed error handling, persistence where required, automated tests, manual Windows verification, accessible states, documented limitations and release notes. A button that only animates or a slider that only changes local UI state is not a completed feature.

19. Implementation and delivery plan

19.1 All-in product scope, dependency-aware execution

The complete product is the target. The following order is an engineering dependency sequence, not a reduction in scope. Codex should work through the entire plan, keeping the full specification visible and avoiding a prototype-only stopping point.

Workstream	Deliverables	Exit condition
A. Foundation	Tauri app, repo conventions, capability model, DB/migrations, logging, CI, installer skeleton.	Clean Windows build, single instance, app lifecycle.
B. Native island	Transparent topmost window, per-region hit-test, geometry, monitor/DPI, hover/focus/shortcut.	Stable overlay across supported Windows configurations.
C. Design system	Visual tokens, shell, all decks, transitions, Milo, responsive layouts.	Complete polished UI with real data interfaces.
D. System adapters	Power, Core Audio, brightness, radios, settings links.	Verified on hardware matrix.
E. Productivity	Tasks, notes, reminders, timers, alarms, notification bridge and reconciliation.	Persistent behavior verified across restart/sleep.
F. Dropzone	Native drop target, OLE drag source, file references, previews, persistence.	External drag-out verified.
G. Music	GSMTC, artwork, controls, session selection, lyrics provider adapter.	Supported players and provider fallback verified.
H. Production	Performance, security, accessibility, installer, signing, updater, docs.	Release acceptance checklist passed.

19.2 High-risk technical spikes (must be resolved early)

- Transparent Tauri window + region-aware hit-testing under Windows 11 and mixed DPI.
- Native OLE drag-out from a Tauri-rendered item to Explorer and a browser upload target.
- AppNotificationManager registration/activation and scheduled notifications from unpackaged app distribution.
- Windows Radio API access and permission flow from Rust/WinRT.
- Internal brightness WMI and external DDC/CI behavior on representative devices.
- GSMTC session and thumbnail behavior across common players.

19.3 Risk register

Risk	Impact	Mitigation
Transparent-window hit-test quirks	High	Native WM_NCHITTEST prototype; keep bounds compact; regression matrix.
OLE drag-out integration	High	Dedicated STA OLE implementation and early target-app tests.
Notification bridge complexity	High	App SDK unpackaged POC; isolate helper if required; schedule limitations documented.
Radio permission/policy blocks	Medium	Read-only status still works; explicit permission and OS settings fallback.
Brightness driver inconsistency	Medium	Capability detection; per-display adapter; no fake control.
Lyrics data rights/availability	Medium	Provider abstraction; legal/terms review; graceful no-lyrics state.
Idle battery usage	High	Event-driven architecture; measure hidden/idle traces.
WebView2/runtime differences	Medium	Evergreen runtime, clean VM testing and runtime recovery guidance.

19.4 Delivery rule

Codex must not interpret the workstream table as permission to omit a deck. If a native feature cannot be completed because of a platform limitation, it must provide the best truthful supported fallback, document the limitation, preserve the UI's disabled/unavailable state and continue with the rest of the product.

20. Codex operating instructions

20.1 Role

Codex is acting as lead desktop application engineer, systems integrator and implementation owner. It must inspect the repository, maintain architecture, implement all required product areas and verify behavior on Windows. It must not merely generate mock UI.

20.2 Mandatory engineering rules

1. Read this specification and inspect the repository before changing files.
2. Maintain a requirements-to-implementation checklist; mark each item implemented, tested, blocked or deferred with a reason.
3. Use TypeScript strict mode and modular React components.
4. Keep native Windows APIs in dedicated Rust adapter modules.
5. Use typed Tauri commands/events; no arbitrary shell command bridge.
6. Use SQLite migrations and preserve user data during upgrades.
7. Use genuine native OLE drag-out for Dropzone.
8. Use Windows App SDK AppNotificationManager or a documented native bridge for scheduled notifications.
9. Use actual Windows state for system controls; never fake success.
10. Add tests for every business rule and native adapter that can be tested.
11. Run formatting, linting, type checks, Cargo tests and Windows integration checks after meaningful changes.
12. Do not silently remove requirements to make a build pass.
13. Record all unsupported hardware/platform conditions and provide UI fallbacks.
14. Keep secrets, signing keys and credentials out of source control.

20.3 Suggested master prompt for Codex

```
You are the lead engineer building MILO, a complete Windows desktop
application. Read the attached MILO Full Technical Requirements
Document v2.0 in full before implementation.

The product is not a website. It is a persistent, top-centre,
hover-expandable Windows desktop island built with Tauri 2,
Rust, React, TypeScript, Motion, Zustand and SQLite.

The complete release scope includes:
- Native floating island, window geometry, focus and hit-testing
- System deck: time, battery, volume, brightness, Wi-Fi,
  Bluetooth and Energy Saver status
- Productivity deck: tasks, lists, reminders, timers, alarms,
  notes and native Windows notifications
- Dropzone: file intake, persistent references and genuine
  native Windows OLE drag-out
- Music deck: GSMTC sessions, metadata, artwork, playback
  controls, timeline and provider-backed lyrics
- Animated Milo companion, visible only when expanded
- Settings, startup behavior, global shortcut, tray access,
  multi-monitor support, local persistence, installer,
  code signing and signed updates

Do not replace native functionality with mock controls.
Do not claim a feature works until it has been tested.
Use Windows APIs through isolated Rust adapters.
Use a native notification bridge if AppNotificationManager
cannot be integrated reliably from the Rust process.
Use a genuine OLE drag source for file drag-out.
Treat hardware and permission limitations honestly.

Maintain a traceable checklist against every MUST requirement.
Implement the entire product, respecting dependencies and
technical risk. Do not stop after a visual prototype.
At every milestone, run tests, fix failures, document any
platform limitation and continue until the release acceptance
criteria are satisfied.

Preserve the MILO visual identity: dark smoked glass,
pearlescent gradients, luminous violet/icy-blue accents,
soft organic curves, restrained ambient light and smooth
transitions. Avoid generic dashboard styling.
```

20.4 Repository documentation

- README.md — setup, run, build and troubleshooting.
- ARCHITECTURE.md — boundaries, native adapters, event flow and threading.
- REQUIREMENTS_TRACEABILITY.md — every requirement mapped to code/tests/status.
- WINDOWS_INTEGRATIONS.md — API choices, permissions, supported hardware and fallbacks.
- DATA_AND_PRIVACY.md — stored data, network calls, retention and deletion.
- RELEASE.md — signing, packaging, update manifest and release procedure.

Appendix A. Proposed project structure

```
milo/
■■■ src/
■   ■■■ app/
■   ■   ■■■ App.tsx
■   ■   ■■■ providers/
■   ■   ■■■ lifecycle/
■   ■■■ components/
■   ■   ■■■ island/
■   ■   ■■■ navigation/
■   ■   ■■■ mascot/
■   ■   ■■■ ui/
■   ■■■ decks/
■   ■   ■■■ system/
■   ■   ■■■ productivity/
■   ■   ■■■ dropzone/
■   ■   ■■■ music/
■   ■■■ features/
■   ■   ■■■ reminders/
■   ■   ■■■ timers/
■   ■   ■■■ alarms/
■   ■   ■■■ tasks/
■   ■   ■■■ lyrics/
■   ■■■ hooks/
■   ■■■ services/
■   ■■■ stores/
■   ■■■ types/
■   ■■■ styles/
■   ■■■ utils/
■■■ src-tauri/
■   ■■■ src/
■   ■   ■■■ main.rs
■   ■   ■■■ lib.rs
■   ■   ■■■ commands/
■   ■   ■■■ events/
■   ■   ■■■ core/
■   ■   ■■■ services/
■   ■   ■■■ windowing/
■   ■   ■■■ persistence/
■   ■   ■■■ models/
■   ■   ■■■ integrations/
■   ■       ■■■ windows/
■   ■           ■■■ power.rs
■   ■           ■■■ audio.rs
■   ■           ■■■ brightness.rs
■   ■           ■■■ radios.rs
■   ■           ■■■ media.rs
■   ■           ■■■ notifications.rs
■   ■           ■■■ ole_drag.rs
■   ■           ■■■ hit_test.rs
■   ■           ■■■ displays.rs
■   ■■■ capabilities/
■   ■■■ migrations/
■   ■■■ icons/
■   ■■■ tauri.conf.json
■■■ notification-bridge/       # only if native App SDK bridge is needed
■■■ tests/
■   ■■■ frontend/
■   ■■■ rust/
■   ■■■ windows-integration/
■■■ docs/
■■■ package.json
■■■ pnpm-lock.yaml
■■■ README.md
```

This is a recommended structure, not a requirement to create empty directories up front. The architecture and ownership boundaries matter more than exact filenames.

Appendix B. Source register and research notes

The following primary documentation was consulted for platform behavior and implementation choices. Links are embedded for direct engineering reference. API availability and behavior should be rechecked against the Windows SDK and Tauri versions pinned at implementation time.

[S01] Microsoft WebView2 distribution
https://learn.microsoft.com/en-us/microsoft-edge/webview2/concepts/distribution
Runtime must be present; Evergreen and Fixed Version deployment options.

[S02] Microsoft Windows notifications overview
https://learn.microsoft.com/en-us/windows/apps/develop/notifications/
Current API guidance: AppNotificationManager for new Windows App SDK desktop applications.

[S03] Microsoft AppNotificationManager registration
https://learn.microsoft.com/en-us/windows/windows-app-sdk/api/winrt/microsoft.windows.appnotifications.appnotificationmanager.register
Unpackaged registration and activation considerations.

[S04] Microsoft Radio.RequestAccessAsync
https://learn.microsoft.com/en-us/uwp/api/windows.devices.radios.radio.requestaccessasync
Permission flow for radio control; read-only enumeration and user/system denial.

[S05] Microsoft WmiMonitorBrightnessMethods
https://learn.microsoft.com/en-us/windows/win32/wmicoreprov/wmimonitorbrightnessmethods
WMI brightness methods for supported monitors.

[S06] Microsoft GSMTC session manager
https://learn.microsoft.com/en-us/uwp/api/windows.media.control.globalsystemmediatransportcontrolssessionmanager
Current/all media sessions and session-change events.

[S07] Microsoft Win32 drag and drop
https://learn.microsoft.com/en-us/windows/win32/com/drag-and-drop
OLE drag source/target model; DoDragDrop and IDataObject/IDropSource.

[S08] Microsoft scheduled app notifications
https://learn.microsoft.com/en-us/windows/apps/develop/notifications/app-notifications/app-notifications-scheduled
Scheduled delivery window and behavior when computer is off.

[S09] Microsoft Windows power consumption guidance
https://learn.microsoft.com/en-us/windows/apps/performance/power
Minimize background resource use, timers and vsync wakeups.

[S10] Microsoft Rust for Windows
https://learn.microsoft.com/en-us/windows/dev-environment/rust/rust-for-windows
Rust windows crate and Win32/WinRT projections.

[S11] Tauri plugins documentation
https://v2.tauri.app/plugin/
Official plugin ecosystem; validate exact plugin and API versions.

[S12] Microsoft GetSystemPowerStatus
https://learn.microsoft.com/en-us/windows/win32/api/winbase/nf-winbase-getsystempowerstatus
Win32 battery and power status API.

[S13] Microsoft Core Audio EndpointVolume API
https://learn.microsoft.com/en-us/windows/win32/coreaudio/endpointvolume-api
Master endpoint volume, mute and change callbacks.

[S14] LRCLIB API documentation
https://lrclib.net/docs
Lyrics lookup, synced/plain fields, client identification and rate-limit guidance; does not itself establish redistribution rights.

[S15] Microsoft Windows app performance guidance
https://learn.microsoft.com/en-us/windows/apps/develop/performance/
Measure startup and interactions; async work and responsiveness.

[S16] Tauri Windows installer documentation
https://v2.tauri.app/distribute/windows-installer/
Windows MSI/NSIS packaging options.

[S17] Tauri Windows code signing
https://v2.tauri.app/distribute/sign/windows/
Windows signing workflow.

Additional implementation references

- **Tauri 2 official documentation** — https://v2.tauri.app/
- **Tauri capabilities and permissions** — https://v2.tauri.app/security/capabilities/
- **Tauri updater plugin** — https://v2.tauri.app/plugin/updater/
- **Tauri global shortcut plugin** — https://v2.tauri.app/plugin/global-shortcut/
- **Microsoft Core Audio interfaces** — https://learn.microsoft.com/en-us/windows/win32/coreaudio/core-audio-interfaces
- **Microsoft OLE DoDragDrop** — https://learn.microsoft.com/en-us/windows/win32/api/ole2/nf-ole2-dodragdrop
- **Microsoft Windows App SDK notifications for .NET** — https://learn.microsoft.com/en-us/windows/apps/develop/notifications/app-notifications/app-notifications-dotnet
- **Motion for React** — https://motion.dev/
- **React** — https://react.dev/
- **TypeScript** — https://www.typescriptlang.org/docs/

End of document — MILO Full Technical Requirements Document v2.0